UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
SI806 · Desarrollo Adaptativo e Integrado del Software

TALLER 3 · DESARROLLO E INTEGRACIÓN DE UNA API REST

Documentación de la API REST

Luxxito Sports: endpoints, contratos JSON, seguridad y códigos HTTP

Referencia de la API REST propia del proyecto: recursos, reglas de negocio que aplica el servidor, autenticación con JWT y roles, titularidad de reservas, concurrencia y ejemplos de solicitudes y respuestas.

Integrantes

Steven Cadillo
Cliente móvil, pagos, pruebas y demostración

Elvis Arboleda
Base de datos Supabase y despliegue

Juan Medina
Backend, Render e infraestructura

Repositorio en GitHub (rama main)
`https://github.com/Arbolito-ctrl/LuxxitoSports/tree/main`

API pública
`https://luxxito-sports-api.onrender.com/api`

Lima, noviembre de 2026

Índice

Datos generales 2
1. Resumen de endpoints 2
2. Convenciones de respuesta 3
3. Seguridad 3
4. Autenticación 4
5. Sistema y documentación 5
6. Catálogo: complejos y canchas 6
7. Reservas 8
8. Pagos y Ticket QR 10

Datos generales

Tabla 1. *Datos generales de la API.*

Aspecto	Detalle
URL base (producción, Render)	`https://luxxito-sports-api.onrender.com/api`
Swagger UI público (probar la API desde el navegador)	`https://luxxito-sports-api.onrender.com/api/docs`
Contrato OpenAPI	https://luxxito-sports-api.onrender.com/api/openapi.yaml (OpenAPI 3.0.3, versión 1.8.0) · fuente en contracts/openapi.yaml · historial en contracts/CHANGELOG.md
URL base (desarrollo local)	http://localhost:3000/api · emulador Android: http://10.0.2.2:3000/api
Formato	JSON application/json; charset=utf-8 (Swagger UI responde HTML y el contrato YAML)
Autenticación	JWT en Authorization: Bearer <token>, firmado con HMAC-SHA256 (JWT_SECRET), vigente 30 días. Roles JUGADOR y ADMIN.
Contraseñas	hash scrypt con sal aleatoria por usuario (scrypt$<sal>$<hash>); nunca en texto plano.
Persistencia	PostgreSQL en Supabase (7 tablas con RLS), bucket privado comprobantes y bucket público canchas.

Este documento se genera a partir de `docs/API_DOCUMENTATION.md`. Todos los ejemplos de respuesta se obtuvieron llamando a la API. El contrato formal está en `contracts/openapi.yaml` (OpenAPI 3.0.3) y cada endpoint se puede probar en Swagger UI: `https://luxxito-sports-api.onrender.com/api/docs`.

1. Resumen de endpoints

La API tiene 31 endpoints. "Público" no requiere token; "Sesión" requiere un JWT válido; "Titular" exige además que el DNI del token sea el de la reserva (o ser `ADMIN`); "ADMIN" exige el rol de operador.

#	Método	Ruta	Qué hace	Acceso
1	`POST`	`/auth/login`	Inicia sesión y devuelve el JWT	Público
2	`POST`	`/auth/registro`	Crea una cuenta JUGADOR y devuelve el JWT	Público
3	`GET`	`/auth/me`	Perfil del usuario del token	Sesión
4	`POST`	`/auth/recuperar`	Solicita recuperar el acceso por correo	Público
5	`GET`	`/dni/:numero`	Sugiere el nombre asociado a un DNI (registro)	Público, 10/min por IP
6	`GET`	`/health`	Estado de la API y de la base de datos	Público
7	`GET`	`/version`	Última versión publicada del APK	Público
8	`GET`	`/apk/download`	Descarga el APK (redirige a GitHub Releases)	Público
9	`GET`	`/docs`	Swagger UI para probar la API	Público
10	`GET`	`/openapi.yaml`	Contrato OpenAPI	Público
11	`GET`	`/complejos`	Lista complejos deportivos	Público
12	`GET`	`/complejos/:id`	Complejo con sus canchas y extras	Público
13	`GET`	`/canchas`	Lista canchas con filtros	Público
14	`GET`	`/canchas/:id`	Detalle de una cancha	Público
15	`POST`	`/canchas`	Crea una cancha (CRUD: Create)	ADMIN
16	`PUT`	`/canchas/:id`	Actualiza una cancha (CRUD: Update)	ADMIN
17	`POST`	`/canchas/:id/imagen`	Sube la foto de la cancha	ADMIN
18	`GET`	`/canchas/:id/disponibilidad`	Turnos del día, tarifas y stock de extras	Público
19	`POST`	`/reservas/cotizar`	Precio calculado por el servidor	Público

#	Método	Ruta	Qué hace	Acceso
20	`POST`	`/reservas`	Crea una reserva con bloqueo de 20 min (CRUD: Create)	Público
21	`GET`	`/reservas`	Lista reservas (CRUD: Read)	Sesión
22	`GET`	`/reservas/:id`	Detalle de una reserva	Titular
23	`PUT`	`/reservas/:id`	Edita contacto y notas (CRUD: Update)	Titular
24	`DELETE`	`/reservas/:id`	Elimina una reserva no confirmada (CRUD: Delete)	Titular
25	`POST`	`/reservas/:id/cancelar`	Cancela con la política de 24 horas	Titular
26	`POST`	`/pagos/comprobante`	Sube la foto del voucher al bucket privado	Público
27	`POST`	`/reservas/:id/pagos`	Registra el comprobante de pago	Titular
28	`GET`	`/pagos`	Bandeja de pagos por revisar	ADMIN
29	`PATCH`	`/pagos/:id/aprobar`	Aprueba el pago y emite el Ticket QR	ADMIN
30	`PATCH`	`/pagos/:id/rechazar`	Rechaza el pago con un motivo	ADMIN
31	`POST`	`/reservas/verificar-qr`	Valida el Ticket QR en la puerta	ADMIN

Todas las rutas empiezan con `/api`. En Swagger UI (`/api/docs`) cada endpoint tiene un botón *Try it out*; el botón *Authorize* permite pegar el JWT para probar las rutas protegidas.

2. Convenciones de respuesta

Una respuesta correcta trae `"ok": true` y los datos en `data` (los listados añaden `count`). Un error trae siempre el mismo formato:

```
{ "ok": false, "code": "CONFLICT", "message": "Texto para mostrar al usuario." }
```

Código HTTP	code	Cuándo ocurre
`400`	BAD_REQUEST / INVALID_JSON	Datos mal formados: DNI sin 8 dígitos, fecha inválida, JSON roto
`401`	UNAUTHORIZED / INVALID_CREDENTIALS	Falta el token, es inválido o venció; o el login es incorrecto
`403`	`FORBIDDEN`	El rol no alcanza (JUGADOR en ruta de ADMIN) o la reserva es de otra persona
`404`	NOT_FOUND / ROUTE_NOT_FOUND	El recurso o la ruta no existen
`409`	`CONFLICT`	Turno ocupado, dato duplicado, estado que no permite la acción, QR ya usado
`413`	`PAYLOAD_TOO_LARGE`	Imagen mayor al límite
`422`	`BUSINESS_RULE_VIOLATION`	Regla de negocio: fuera de horario, más de 4 horas, monto incorrecto
`429`	`TOO_MANY_REQUESTS`	Demasiadas consultas de DNI desde la misma IP
`502`	STORAGE_ERROR / VERSION_NO_DISPONIBLE	Falló un servicio externo (Storage o GitHub)
`503`	DATABASE_UNAVAILABLE / DNI_LOOKUP_UNAVAILABLE	La base de datos o la consulta de DNI no están disponibles

3. Seguridad

3.1. JWT y roles

Las rutas protegidas exigen `Authorization: Bearer <token>`. El token lleva `id`, `dni`, `rol` y `exp` (30 días):

- Sin token, con firma alterada o vencido: `401 UNAUTHORIZED`.
- Token de `JUGADOR` en una ruta de `ADMIN`: `403 FORBIDDEN`.
- Son solo de `ADMIN`: crear y editar canchas, subir su foto, la bandeja de pagos, aprobar, rechazar y verificar el QR.

3.2. Titularidad de reservas

`GET`, `PUT` y `DELETE /reservas/:id`, `POST /reservas/:id/pagos` y `POST /reservas/:id/cancelar` solo los usa el titular (el DNI del token coincide con `cliente_dni`) o un `ADMIN`. Una reserva ajena responde `403`. En `GET /reservas` un `JUGADOR` solo recibe las suyas, aunque envíe el DNI de otra persona.

3.3. Contraseñas

El registro guarda la contraseña con `scrypt` (sal de 16 bytes, hash de 64 bytes) y el login la compara en tiempo constante. Una cuenta antigua con contraseña en texto plano se migra a hash en su primer inicio de sesión.

3.4. Concurrencia

- Dos pedidos simultáneos por el mismo turno: solo uno crea la reserva; el otro recibe `409`. Lo garantizan una cola por fecha en el servidor y la restricción de exclusión `reservas_sin_solapamiento` de PostgreSQL.
- Aprobar, rechazar, cancelar o validar el QR de una misma reserva a la vez se procesa en orden: el QR se acepta una sola vez.

3.5. Datos protegidos

- La app solo habla con esta API: no lleva claves de Supabase.
- En Supabase, la clave pública (`anon`) solo puede leer el catálogo (complejos, canchas y extras). Reservas, pagos y usuarios, y las funciones `rpc_*`, quedaron cerrados (migración `2026-11-13_cerrar_acceso_publico.sql`).
- La consulta de DNI devuelve solo nombres y apellidos; el token del proveedor vive solo en el servidor.

4. Autenticación

4.1. POST /api/auth/login

Inicia sesión con correo y contraseña. **Acceso:** público.

```
{ "email": "elvis.arboleda@uni.pe", "password": "luxxito2026" }
```

Respuesta 200:

```
{
  "ok": true,
  "message": "Inicio de sesión exitoso.",
  "data": {
    "user": { "id": 1, "nombre": "Elvis Arboleda", "dni": "74125896",
              "telefono": "987654321", "email": "elvis.arboleda@uni.pe",
              "rol": "JUGADOR" },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Errores: `400` si falta el correo o la contraseña; `401 INVALID_CREDENTIALS` ("Correo o contraseña incorrectos.").

4.2. POST /api/auth/registro

Crea una cuenta con rol `JUGADOR` y devuelve el JWT, igual que el login. **Acceso:** público.

```
{ "nombre": "Ana Torres", "dni": "71234567", "telefono": "987654321",
  "email": "ana.torres@uni.pe", "password": "clave2026" }
```

Respuesta 201: `{ "ok": true, "message": "Cuenta creada exitosamente.", "data": { "user": {...}, "token": "..." } }`

Errores:

- `400`: nombre de menos de 3 caracteres, DNI sin 8 dígitos, teléfono vacío, correo mal formado o contraseña de menos de 4 caracteres.
- `409`: "Ya existe una cuenta registrada con ese correo o DNI."

4.3. GET /api/auth/me

Devuelve el perfil del usuario dueño del token. La app lo usa para restaurar la sesión al abrirse. **Acceso:** sesión.

```
{ "ok": true, "data": { "user": { "id": 1, "nombre": "Elvis Arboleda", "dni": "74125896",
  "telefono": "987654321", "email": "elvis.arboleda@uni.pe", "rol": "JUGADOR" } } }
```

Errores: `401` sin token o con token inválido; `404` si el usuario ya no existe.

4.4. POST /api/auth/recuperar

Registra el pedido para recuperar el acceso. **Acceso:** público. En el taller confirma el pedido; no envía un correo real.

```
{ "email": "ana.torres@uni.pe" }
```

Respuesta 200: `{ "ok": true, "message": "Se han enviado las instrucciones de recuperación de contraseña a ana.torres@uni.pe." }`

Errores: `400` si el correo no es válido.

4.5. GET /api/dni/:numero

Sugiere el nombre completo asociado a un DNI para autocompletar el registro. **Acceso:** público, máximo 10 consultas por minuto por IP.

Consulta al proveedor Decolecta y devuelve **solo nombres y apellidos**, los mismos datos que RENIEC muestra gratis en su web. Guarda la respuesta 24 horas en memoria para no gastar la cuota del proveedor. El registro no depende de esta ruta: si falla, el usuario escribe su nombre.

Respuesta 200 (GET /api/dni/46027897):

```
{
  "ok": true,
  "data": { "dni": "46027897", "nombres": "JUAN CARLOS", "apellido_paterno": "PEREZ",
            "apellido_materno": "GARCIA", "nombre_completo": "JUAN CARLOS PEREZ GARCIA" }
}
```

Errores: `400` si no son 8 dígitos; `404` si no se encuentra; `429` (con cabecera `Retry-After`) al pasar el límite; `503 DNI_LOOKUP_UNAVAILABLE` si el servidor no tiene el token o el proveedor no responde.

5. Sistema y documentación

5.1. GET /api/health

Estado de la API y de la base de datos. Render lo usa para saber si el servicio está vivo. **Acceso:** público.

```
{
  "ok": true,
  "service": "Luxxito Sports REST API (SI806 - Taller 3)",
  "db_mode": "POSTGRES_POOL",
  "db_ready": true,
  "api_version": "1.8.0",
  "commit": "95499a33e5d16539e6dc0f11bec784c34c7b1977",
  "apk_download_url": "https://github.com/.../latest/download/luxxito-sports.apk",
  "timestamp": "2026-11-13T18:01:00.413Z"
}
```

`commit` es la versión desplegada en Render. **Errores:** `503 DATABASE_UNAVAILABLE` si la base no responde.

5.2. GET /api/version?canal=latest-apk

Última versión publicada del APK, leída del `version.json` que genera el CI en GitHub Releases. La app la usa para ofrecer la actualización. `canal` acepta `latest-apk` (producción, por defecto) o `dev-apk`. **Acceso:** público.

```
{
  "ok": true,
  "data": { "version": "1.0.49", "build_number": 49, "canal": "latest-apk",
            "commit": "95499a33e5d16539e6dc0f11bec784c34c7b1977",
            "apk_url": "https://github.com/.../latest-apk/luxxito-sports.apk",
            "api_version": "1.8.0" }
}
```

Errores: `502 VERSION_NO_DISPONIBLE` si GitHub no responde.

5.3. GET /api/apk/download

Descarga el APK. Responde `302 Found` y redirige a `https://github.com/Arbolito-ctrl/LuxxitoSports/releases/latest/download/luxxito-sports.apk`. **Acceso:** público.

5.4. GET /api/docs

Página de Swagger UI (HTML) para explorar y probar todos los endpoints desde el navegador: `https://luxxito-sports-api.onrender.com/api/docs`. **Acceso:** público.

5.5. GET /api/openapi.yaml

Entrega el contrato OpenAPI 3.0.3 (`application/yaml`) que usa Swagger UI. Se puede importar en Postman, Swagger Editor o Redocly. **Acceso:** público.

6. Catálogo: complejos y canchas

6.1. GET /api/complejos?distrito=San Miguel

Lista los complejos activos; `distrito` es opcional. **Acceso:** público.

```
{
  "ok": true,
  "count": 1,
  "data": [
    { "id": 1, "nombre": "Luxxito Arena San Miguel", "distrito": "San Miguel",
      "direccion": "Av. La Marina 2450, San Miguel", "telefono": "987654321",
      "hora_apertura": 7, "hora_cierre": 23, "imagen_url": "https://...", "activo": true }
  ]
}
```

6.2. GET /api/complejos/:id

Un complejo con sus `canchas` y sus `extras`. **Acceso:** público.

Errores: `400` si el ID no es un entero positivo; `404` ("No se encontró el complejo deportivo con ID 999.").

6.3. GET /api/canchas

Lista canchas. Filtros opcionales: `complejo_id`, `distrito`, `deporte` y `estado`. **Acceso:** público.

```
{
  "ok": true,
  "count": 2,
  "data": [
    { "id": 1, "complejo_id": 1, "nombre": "Cancha Monumental 1", "deporte": "Futbol 7",
      "superficie": "Grass Sintetico FIFA Quality", "techada": false, "iluminacion": true,
      "precio_hora_diurno": 90, "precio_hora_nocturno": 120, "estado": "ACTIVA",
      "complejo_nombre": "Luxxito Arena San Miguel", "distrito": "San Miguel",
      "direccion": "Av. La Marina 2450, San Miguel" }
  ]
}
```

6.4. GET /api/canchas/:id

Detalle de una cancha, con los mismos campos que el listado. **Acceso:** público. **Errores:** `400` y `404`.

6.5. POST /api/canchas

Crea una cancha. **Acceso:** `ADMIN`.

```
{ "complejo_id": 1, "nombre": "Cancha Docs 9", "deporte": "Futbol 5",
  "superficie": "Grass sintético", "techada": false, "iluminacion": true,
  "precio_hora_diurno": 80, "precio_hora_nocturno": 110 }
```

Respuesta 201: `{ "ok": true, "message": "Cancha creada exitosamente.", "data": { "id": 7, ..., "estado": "ACTIVA" } }`

Reglas y errores:

- `400`: nombre de menos de 3 caracteres, precio diurno menor o igual a 0, o nocturno menor que el diurno.
- `404`: el complejo no existe.
- `409`: ya existe una cancha con ese nombre en el complejo (sin distinguir mayúsculas).
- `401` sin token; `403` con rol `JUGADOR`.

6.6. PUT /api/canchas/:id

Actualiza solo los campos enviados: nombre, tarifas, estado (`ACTIVA`, `MANTENIMIENTO` o `INACTIVA`) o características. **Acceso:** `ADMIN`.

```
{ "precio_hora_nocturno": 120, "estado": "MANTENIMIENTO" }
```

Respuesta 200: `{ "ok": true, "message": "Cancha actualizada exitosamente.", "data": {...} }`. **Errores:** los mismos que al crear.

6.7. POST /api/canchas/:id/imagen

Sube la foto de la cancha al bucket público `canchas` y guarda su `imagen_url`. **Acceso:** `ADMIN`.

```
{ "mime_type": "image/jpeg", "contenido_base64": "/9j/4AAQSkZJRg..." }
```

Respuesta 200: `{ "ok": true, "message": "Foto de la cancha actualizada.", "data": { ..., "imagen_url": "https://..." } }`

Errores: `400` si el formato no es JPEG, PNG o WEBP o falta la imagen; `413`/`422` si supera 5 MB; `404` si la cancha no existe; `502` si falla Storage.

6.8. GET /api/canchas/:id/disponibilidad?fecha=2026-11-19

Regla de negocio 1. Devuelve los turnos de 1 hora del día dentro del horario del complejo, con su tarifa y su estado, y el stock libre de cada extra. Si no se envía `fecha`, usa el día actual de Lima. **Acceso:** público.

- Tarifa `DIURNA` antes de las 18:00 y `NOCTURNA` desde las 18:00.
- Estado del turno: `DISPONIBLE`, `BLOQUEADA` (otra persona lo está pagando), `OCUPADA` (confirmado), `MANTENIMIENTO` o `PASADA`.

```
{
  "ok": true,
  "data": {
    "cancha": { "id": 1, "nombre": "Cancha Monumental 1", "precio_hora_diurno": 90,
                "precio_hora_nocturno": 120, "estado": "ACTIVA", ... },
    "complejo": { "id": 1, "nombre": "Luxxito Arena San Miguel", "hora_apertura": 7,
                  "hora_cierre": 23, ... },
    "fecha": "2026-11-19",
    "horario_operacion": { "apertura": 7, "cierre": 23 },
    "turnos": [
      { "hora_inicio": 17, "hora_fin": 18, "etiqueta": "17:00 - 18:00",
        "tarifa_tipo": "DIURNA", "precio": 90, "estado": "DISPONIBLE" },
      { "hora_inicio": 19, "hora_fin": 20, "etiqueta": "19:00 - 20:00",
        "tarifa_tipo": "NOCTURNA", "precio": 120, "estado": "OCUPADA" }
    ],
    "extras_disponibles": [
      { "id": 1, "nombre": "Juego de Chalecos (x12)", "precio_unitario": 15,
        "stock_total": 4, "stock_disponible": 4 }
    ]
  }
}
```

Errores: `400` si la fecha no es `YYYY-MM-DD`; `404` si la cancha no existe.

7. Reservas

7.1. POST /api/reservas/cotizar

Regla de negocio 2. El servidor calcula el precio; la app nunca envía montos. Valida la cancha, el horario de atención, la duración máxima de 4 horas y el stock de los extras en ese intervalo. No guarda nada. **Acceso:** público.

```
{ "cancha_id": 1, "fecha": "2026-11-19", "hora_inicio": 19, "hora_fin": 21,
  "extras": [ { "extra_id": 1, "cantidad": 2 } ] }
```

Respuesta 200:

```
{
  "ok": true,
  "data": {
    "cancha_id": 1, "cancha_nombre": "Cancha Monumental 1",
    "complejo_nombre": "Luxxito Arena San Miguel", "fecha": "2026-11-19",
    "hora_inicio": 19, "hora_fin": 21, "duracion_horas": 2,
    "subtotal_cancha": 240, "subtotal_extras": 30, "total": 270, "monto_sena_50": 135,
    "desglose_horas": [
      { "hora": "19:00 - 20:00", "tipo": "NOCTURNA", "precio": 120 },
      { "hora": "20:00 - 21:00", "tipo": "NOCTURNA", "precio": 120 }
    ],
    "desglose_extras": [
      { "extra_id": 1, "nombre": "Juego de Chalecos (x12)", "cantidad": 2,
        "precio_unitario": 15, "subtotal": 30 }
    ]
  }
}
```

Errores:

- `400`: datos mal formados o un extra de otro complejo.
- `404`: la cancha no existe.
- `409`: cancha no activa o stock insuficiente.
- `422`: fuera del horario del complejo o más de 4 horas ("La duración máxima por reserva es de 4 horas consecutivas.").

7.2. POST /api/reservas

Regla de negocio 3. Crea la reserva en `PENDIENTE_PAGO` y retiene el turno 20 minutos (`expires_at`). Repite las validaciones de la cotización y además rechaza horas pasadas (hora de Lima) y turnos que se cruzan con otra reserva activa. **Acceso:** público.

```
{
  "cancha_id": 1, "fecha": "2026-11-19", "hora_inicio": 19, "hora_fin": 21,
  "cliente_nombre": "Elvis Arboleda", "cliente_dni": "74125896",
  "cliente_telefono": "987000111", "cliente_email": "elvis.arboleda@uni.pe",
  "notas": "Pichanga UNI", "extras": [ { "extra_id": 1, "cantidad": 2 } ]
}
```

Respuesta 201:

```
{
  "ok": true,
  "message": "Reserva creada con bloqueo temporal de 20 minutos.",
  "data": {
    "id": 1, "cancha_id": 1, "cancha_nombre": "Cancha Monumental 1",
    "complejo_nombre": "Luxxito Arena San Miguel", "cliente_nombre": "Elvis Arboleda",
    "cliente_dni": "74125896", "fecha": "2026-11-19", "hora_inicio": 19, "hora_fin": 21,
    "subtotal_cancha": 240, "subtotal_extras": 30, "total": 270,
    "monto_pagado": 0, "monto_devolucion": 0, "estado": "PENDIENTE_PAGO",
    "codigo_qr": null, "expires_at": "2026-11-13T18:21:01.475Z",
    "extras": [ { "extra_id": 1, "nombre": "Juego de Chalecos (x12)", "cantidad": 2,
                  "precio_unitario": 15, "subtotal": 30 } ],
    "pagos": []
  }
}
```

Errores:

- `400`: nombre de menos de 3 caracteres, DNI sin 8 dígitos, teléfono o correo inválidos.
- `409`: "El turno de 20:00 a 21:00 ya se encuentra ocupado o bloqueado (Reserva #1 en estado PENDIENTE_PAGO)." o un pedido simultáneo por el mismo turno.
- `422`: fecha u hora pasada, fuera de horario o más de 4 horas.

7.3. GET /api/reservas

CRUD: Read. Lista reservas con sus `extras` y `pagos`. Filtros opcionales: `estado`, `cliente_dni` (solo `ADMIN`) y `fecha`. Antes de responder marca como `VENCIDA` las que pasaron sus 20 minutos sin pago.

Acceso: sesión.

Respuesta 200: `{ "ok": true, "count": 1, "data": [ { ...reserva completa... } ] }`.
Errores: `401`.

7.4. GET /api/reservas/:id

Detalle completo de una reserva, con extras y pagos. **Acceso:** titular o `ADMIN`. **Errores:** `400`, `401`, `403` y `404`.

7.5. PUT /api/reservas/:id

CRUD: Update. Edita solo el contacto (`cliente_nombre`, `cliente_telefono`, `cliente_email`) y las `notas`. La cancha, la fecha y el horario no se cambian porque alterarían el precio. **Acceso:** titular o `ADMIN`.

```
{ "cliente_telefono": "999888777", "notas": "Llevamos pelota" }
```

Respuesta 200: `{ "ok": true, "message": "Datos de la reserva actualizados correctamente.", "data": {...} }`

Errores: `400` si el correo no es válido; `409` si la reserva está `CANCELADA` o `VENCIDA`; `401`, `403` y `404`.

7.6. DELETE /api/reservas/:id

CRUD: Delete. Borra la reserva y libera su turno y sus extras. Una reserva `CONFIRMADA` no se borra: tiene dinero pagado y debe cancelarse. **Acceso:** titular o `ADMIN`.

Respuesta 200: `{ "ok": true, "message": "Reserva #2 eliminada correctamente.", "deleted_id": 2 }`

Errores: `409` si está `CONFIRMADA`; `401`, `403` y `404`.

7.7. POST /api/reservas/:id/cancelar

Regla de negocio 6. Cancela aplicando la política de 24 horas, anula el Ticket QR y libera el turno y los extras. **Acceso:** titular o `ADMIN`.

- Con 24 horas o más de anticipación: devolución del 100 % de lo pagado (`REEMBOLSO_100_MAYOR_24H`).
- Con menos de 24 horas: se retiene el 50 % como penalidad (`RETENCION_50_MENOR_24H`).
- Sin pagos previos: `SIN_PAGOS_PREVIOS`.

```
{ "motivo": "Ya no podré asistir" }
```

Respuesta 200:

```
{
  "ok": true,
  "message": "Reserva cancelada aplicando la política de reembolso de 24 horas.",
  "data": {
    "reserva": { "id": 1, "estado": "CANCELADA", "codigo_qr": null,
                 "motivo_cancelacion": "Ya no podré asistir",
                 "monto_devolucion": 270, ... },
    "politica_aplicada": "REEMBOLSO_100_MAYOR_24H",
    "horas_anticipacion": 145.32,
    "monto_pagado": 270,
    "penalidad": 0,
    "monto_devolucion": 270
  }
}
```

Errores: `409` si ya estaba cancelada; `401`, `403` y `404`.

8. Pagos y Ticket QR

El flujo de pago es manual: el jugador paga por Yape, Plin o transferencia, sube la foto del voucher y registra el pago; el `ADMIN` lo revisa en la bandeja y lo aprueba (la reserva queda `CONFIRMADA` con su Ticket QR) o lo rechaza con un motivo.

8.1. POST /api/pagos/comprobante

Sube la foto del voucher (JPEG, PNG o WEBP, máximo 5 MB, en base64) al bucket **privado** `comprobantes`. Devuelve `comprobante_url`, que luego se envía al registrar el pago. **Acceso:** público.

```
{ "reserva_id": 1, "mime_type": "image/jpeg", "contenido_base64": "/9j/4AAQSkZJRg..." }
```

Respuesta 201:

```
{
  "ok": true,
  "message": "Comprobante almacenado en el bucket privado comprobantes.",
  "data": {
    "bucket": "comprobantes",
    "path": "reserva-1/voucher-1794592861492.jpg",
    "comprobante_url": "supabase://comprobantes/reserva-1/voucher-1794592861492.jpg",
    "mime_type": "image/jpeg",
    "size_bytes": 184233,
    "almacenado": true
  }
}
```

Errores: `400` si el formato no se permite o falta la imagen; `404` si la reserva no existe; `409` si está `CANCELADA` o `VENCIDA`; `413`/`422` si supera el tamaño; `502` si falla Storage.

8.2. POST /api/reservas/:id/pagos

Regla de negocio 4. Registra el comprobante: el pago queda `PENDIENTE_REVISION` y la reserva pasa a `EN_REVISION`, lo que la protege de vencer mientras el `ADMIN` revisa. **Acceso:** titular o `ADMIN`.

- `metodo`: `YAPE`, `PLIN` o `TRANSFERENCIA`. `modalidad`: `TOTAL` o `SENA_50` (adelanto del 50 % en el primer pago).
- `monto` es opcional: si no se envía, el servidor usa el que corresponde a la modalidad.
- `comprobante_url` debe ser un voucher subido para esta misma reserva.

```
{ "metodo": "YAPE", "modalidad": "TOTAL", "numero_operacion": "YAPE-9948102",
  "comprobante_url": "supabase://comprobantes/reserva-1/voucher-1794592861492.jpg" }
```

Respuesta 201:

```
{
  "ok": true,
  "message": "Comprobante registrado exitosamente. Reserva protegida en revisión.",
  "data": {
    "pago": { "id": 1, "reserva_id": 1, "metodo": "YAPE", "modalidad": "TOTAL",
              "monto": 270, "numero_operacion": "YAPE-9948102",
              "estado": "PENDIENTE_REVISION", "observacion": null },
    "reserva": { "id": 1, "estado": "EN_REVISION", "total": 270, "monto_pagado": 0, ... }
  }
}
```

Errores:

- `400`: método o modalidad inválidos, número de operación de menos de 4 caracteres, o voucher de otra reserva.
- `409`: reserva cancelada o vencida, ya pagada, o con otro comprobante en revisión.
- `422`: el monto no coincide con la modalidad.
- `401`, `403` y `404`.

8.3. GET /api/pagos?estado=PENDIENTE_REVISION

Bandeja del operador. Cada pago trae el resumen de su reserva y `comprobante_url_firmada`, un enlace temporal (1 hora) para ver el voucher del bucket privado. `estado` acepta `PENDIENTE_REVISION`, `APROBADO`, `RECHAZADO` o `REEMBOLSADO`. **Acceso:** `ADMIN`.

```
{
  "ok": true,
  "count": 1,
  "data": [
    {
      "id": 1, "reserva_id": 1, "metodo": "YAPE", "modalidad": "TOTAL", "monto": 270,
      "numero_operacion": "YAPE-9948102", "estado": "PENDIENTE_REVISION",
      "comprobante_url_firmada": "https://...supabase.co/storage/v1/object/sign/...",
      "reserva": { "id": 1, "estado": "EN_REVISION",
                   "cancha_nombre": "Cancha Monumental 1",
                   "cliente_nombre": "Elvis Arboleda", "cliente_dni": "74125896",
                   "cliente_telefono": "999888777", "fecha": "2026-11-19",
                   "hora_inicio": 19, "hora_fin": 21, "total": 270, "monto_pagado": 0 }
    }
  ]
}
```

Errores: `400` si el estado no es válido; `401`; `403` con rol `JUGADOR`.

8.4. PATCH /api/pagos/:id/aprobar

Regla de negocio 5. Aprueba el pago: suma el monto a la reserva, la deja `CONFIRMADA` y genera su Ticket QR (`LUXXITO-QR-<id>-<huella>`). Es **idempotente**: aprobar de nuevo el mismo pago no vuelve a sumar el monto y responde `idempotent: true`. **Acceso:** `ADMIN`. No lleva cuerpo.

```
{
  "ok": true,
  "message": "Pago aprobado exitosamente. Reserva confirmada con Ticket QR.",
  "data": {
    "pago": { "id": 2, "estado": "APROBADO",
              "observacion": "Comprobante verificado y conciliado." },
    "reserva": { "id": 1, "estado": "CONFIRMADA", "monto_pagado": 270,
                 "codigo_qr": "LUXXITO-QR-1-4B4316B6A0", ... },
    "idempotent": false
  }
}
```

Errores: `409` si la reserva está cancelada o vencida, o si el pago fue rechazado; `400`, `401`, `403` y `404`.

8.5. PATCH /api/pagos/:id/rechazar

Rechaza el pago con un motivo obligatorio (mínimo 5 caracteres) que el jugador verá en la app. Si la reserva estaba `EN_REVISION`, vuelve a `PENDIENTE_PAGO` con 20 minutos nuevos para reenviar el comprobante. También es idempotente. **Acceso:** `ADMIN`.

```
{ "observacion": "El número de operación no coincide con el voucher" }
```

Respuesta 200:

```
{
  "ok": true,
  "message": "Pago rechazado. El jugador puede enviar un nuevo comprobante.",
  "data": {
    "pago": { "id": 1, "estado": "RECHAZADO",
              "observacion": "El número de operación no coincide con el voucher" },
    "reserva": { "id": 1, "estado": "PENDIENTE_PAGO",
                 "expires_at": "2026-11-13T18:21:01.505Z", ... },
    "idempotent": false
  }
}
```

Errores: `400` sin motivo; `409` si el pago ya fue aprobado o reembolsado; `401`, `403` y `404`.

8.6. POST /api/reservas/verificar-qr

Cierra la regla 5: el operador escanea en la puerta el Ticket QR emitido al aprobar el pago. Se acepta solo si pertenece a una reserva `CONFIRMADA` y no se usó antes; al aceptarlo se registra la hora de ingreso.
Acceso: `ADMIN`.

```
{ "codigo_qr": "LUXXITO-QR-1-4B4316B6A0" }
```

Respuesta 200:

```
{
  "ok": true,
  "message": "Ticket QR verificado exitosamente. Ingreso registrado.",
  "data": {
    "verificado": true,
    "estado_ingreso": "INGRESO_REGISTRADO",
    "verificado_en": "2026-11-13T18:01:01.518Z",
    "reserva": { "id": 1, "estado": "CONFIRMADA",
                 "cliente_nombre": "Elvis Arboleda", ... }
  }
}
```

Errores:

- `400`: el código no empieza con `LUXXITO-QR-`.
- `404`: el código no pertenece a ninguna reserva activa.
- `409`: la reserva fue cancelada (QR revocado), no está confirmada, o el ticket ya se usó ("Este Ticket QR ya fue utilizado para ingresar (13/11, 01:01 p. m.).").
- `401` y `403`.